

Experiment No: 1

Cryptography in Blockchain: Merkle Root Tree Hash

Jay Kerkar

Roll No: 16

Class: D20A, Batch A

Aim

Cryptography in Blockchain, Merkle Root Tree Hash.

Lab Objectives

To explore Blockchain concepts.

Lab Outcomes (LO)

Creating Cryptographic Hash using Merkle Tree (LO1).

Tools & Libraries Used

- Python Libraries: `hashlib`

Tasks Performed

1. **Hash Generation using SHA-256:** Developed a Python program to compute a SHA-256 hash for any given input string using the `hashlib` library.
2. **Target Hash Generation with Nonce:** Created a program to generate a hash code by concatenating a user input string and a nonce value to simulate the mining process.
3. **Proof-of-Work Puzzle Solving:** Implemented a program to find the nonce that, when combined with a given input string, produces a hash starting with a specified number of leading zeros.
4. **Merkle Tree Construction:** Built a Merkle Tree from a list of transactions by recursively hashing pairs of transaction hashes, doubling up the last node if needed, and generated the Merkle Root hash for blockchain transaction integrity. The transaction list was updated with valid sample transactions, each transaction was hashed before being combined in the loop, and all intermediate hashes were printed during construction.

Program and Output

Program #1: SHA-256 Hash of a Given String

```
1 import hashlib
2
3 def create_hash(string):
4     # Create a hash object using SHA-256 algorithm
5     hash_object = hashlib.sha256()
6     # Convert the string to bytes and update the hash object
7     hash_object.update(string.encode('utf-8'))
8     # Get the hexadecimal representation of the hash
```

```
9     hash_string = hash_object.hexdigest()
10     # Return the hash string
11     return hash_string
12
13 # Example usage
14 input_string = input("Enter a string: ")
15 hash_result = create_hash(input_string)
16 print("Hash:", hash_result)
```

Output:

```
1 Enter a string: a
2 Hash: ca978112ca1bbdcafac231b39a23dc4da786eff8147c4e72b9807785afee48
```

Program #2: Target Hash from Input String and Nonce

```
1 import hashlib
2
3 # Get user input
4 input_string = input("Enter a string: ")
5 nonce = input("Enter the nonce: ")
6
7 # Concatenate the string and nonce
8 hash_string = input_string + nonce
9
10 # Calculate the hash using SHA-256
11 hash_object = hashlib.sha256(hash_string.encode('utf-8'))
12 hash_code = hash_object.hexdigest()
13
14 # Print the hash code
15 print("Hash Code:", hash_code)
```

Output:

```
1 Enter a string: A
2 Enter the nonce: 1
3 Hash Code: 16a36e86f6fed5d465ff332511a0ce1a863b55d364b25a7cdaa25db19abf964
```

Program #3: Solving the Proof-of-Work Puzzle (Golden Nonce)

```
1 import hashlib
2
3 def find_nonce(input_string, num_zeros):
4     nonce = 0
5     hash_prefix = '0' * num_zeros
6
7     while True:
8         # Concatenate the string and nonce
9         hash_string = input_string + str(nonce)
10        # Calculate the hash using SHA-256
11        hash_object = hashlib.sha256(hash_string.encode('utf-8'))
12        hash_code = hash_object.hexdigest()
13
14        # Check if the hash code has the required number of leading zeros
15        if hash_code.startswith(hash_prefix):
16            print("Hash:", hash_code)
17            return nonce
18
19        nonce += 1
20
21 # Get user input
```

```

22 input_string = "A"
23 num_zeros = 1
24
25 # Find the expected nonce
26 expected_nonce = find_nonce(input_string, num_zeros)
27
28 # Print the expected nonce
29 print("Input String:", input_string)
30 print("Leading Zeros:", num_zeros)
31 print("Expected Nonce:", expected_nonce)

```

Output:

```

1 Hash: 06663975f75f189f1d70bd14d8f22df264cd6a5c7575d6875183d0ec76432fb
2 Input String: A
3 Leading Zeros: 1
4 Expected Nonce: 9

```

Program #4: Constructing a Merkle Tree Root Hash

The transaction list was updated with the sample transactions T1–T5. Each transaction is hashed before being combined in the loop, and all intermediate hashes at every level are printed.

```

1 import hashlib
2
3 def hash_data(data):
4     # Compute SHA-256 hash of the given string
5     return hashlib.sha256(data.encode('utf-8')).hexdigest()
6
7 def build_merkle_tree(transactions):
8     if len(transactions) == 0:
9         return None
10
11     # Hash each transaction before combining
12     transactions = [hash_data(tx) for tx in transactions]
13     print("Transaction Hashes:")
14     for i, h in enumerate(transactions):
15         print(f" T{i+1} Hash: {h}")
16
17     if len(transactions) == 1:
18         return transactions[0]
19
20     level = 1
21     # Recursive construction of the Merkle Tree
22     while len(transactions) > 1:
23         if len(transactions) % 2 != 0:
24             transactions.append(transactions[-1])
25
26         new_transactions = []
27         print(f"\nLevel {level} Hashes:")
28         for i in range(0, len(transactions), 2):
29             combined = transactions[i] + transactions[i + 1]
30             hash_combined = hash_data(combined)
31             print(f" Combining {transactions[i][:8]}... + "
32                   f"{transactions[i+1][:8]}... -> {hash_combined}")
33             new_transactions.append(hash_combined)
34
35         transactions = new_transactions
36         level += 1
37
38     return transactions[0]
39
40 # Example usage - sample transactions

```

```

41 transactions = [
42     'Alice -> Bob : $200',
43     'Bob -> Dave : $500',
44     'Dave -> Eve : $100',
45     'Eve -> Alice : $300',
46     'Roo -> Bob : $50'
47 ]
48
49 merkle_root = build_merkle_tree(transactions)
50 print("\nMerkle Root:", merkle_root)

```

Output:

```

1 Transaction Hashes:
2   T1 Hash: f817ce0bddbfa417eba4ae519a8b864ba7b1eed286e10c4b92086cc71327976
3   T2 Hash: 768890d3b58d4d6a17673e6f750a0145f546557c9529d258750e1ca0cdbb5b4
4   T3 Hash: 43dd5729f00e01eea73a858d02f9c714c882bab801de203c1314d6318d6890d
5   T4 Hash: cba341d7965fff73181a2a6579799dc9644e84380fa9e14f12ebf78c70316a8
6   T5 Hash: cfe1dc26c7c3e26c19aed0552bd4b9db2cc797a65a3a26316f47ea95cb1b58a
7
8 Level 1 Hashes:
9   Combining f817ce0b... + 768890d3... -> 471608
10      d3d6f2a642ff3a84dfaeeaebb1c271a9aed5b37a82d61c4784558e80e
11   Combining 43dd5729... + cba341d7... ->
12      a61c504d72e6ce69b2ec8791ff7469cc004704f727f8dbbf420fadbc4e4c0de
13   Combining cfe1dc26... + cfe1dc26... -> 314
14      ca746b3d4fe63a1e0b939ec3042df6270de7703aa0eb7e699a3f2c2e7c18
15
16 Level 2 Hashes:
17   Combining 471608d3... + a61c504d... ->
18      ae847393c3dff60e1d95cfdbb1c1c886d0bf7cbd26410bcc2469e023e94cb8e
19   Combining 314ca746... + 314ca746... -> 6
20      bb2c8b4fd23658c0b716069761d9811ced49f7e4f2f736b493fe0c0a96b4f4
21
22 Level 3 Hashes:
23   Combining ae847393... + 6bb2c8b4... -> 96
24      ee90df2272d24c0bcf4c67e152c257dc45bce4560221125b0750cead7b27b
25
26 Merkle Root: 96ee90df2272d24c0bcf4c67e152c257dc45bce4560221125b0750cead7b27b

```

Conclusion

This study demonstrates fundamental cryptographic concepts and their applications within blockchain technology:

- **SHA-256 Hashing** provides a secure and unique fingerprint for any input, ensuring data integrity.
- **Nonce and Target Hashing** allow the simulation of proof-of-work consensus, emphasizing the difficulty of mining blocks by meeting hash conditions (leading zeros).
- **Proof-of-Work Puzzle Solving** shows how mining requires computational effort to find a nonce making the hash satisfy network difficulty.
- **Merkle Tree Construction** efficiently summarizes and validates large numbers of transactions with a single Merkle root hash, which is essential in blockchain for fast verification and ensuring the integrity of data blocks.

Together, these implementations capture the essence of how cryptography underpins blockchain's security, immutability, and efficiency, especially through hash functions and structures like Merkle Trees that enable trustless and decentralized transaction management.